

Reading the ACE introduction

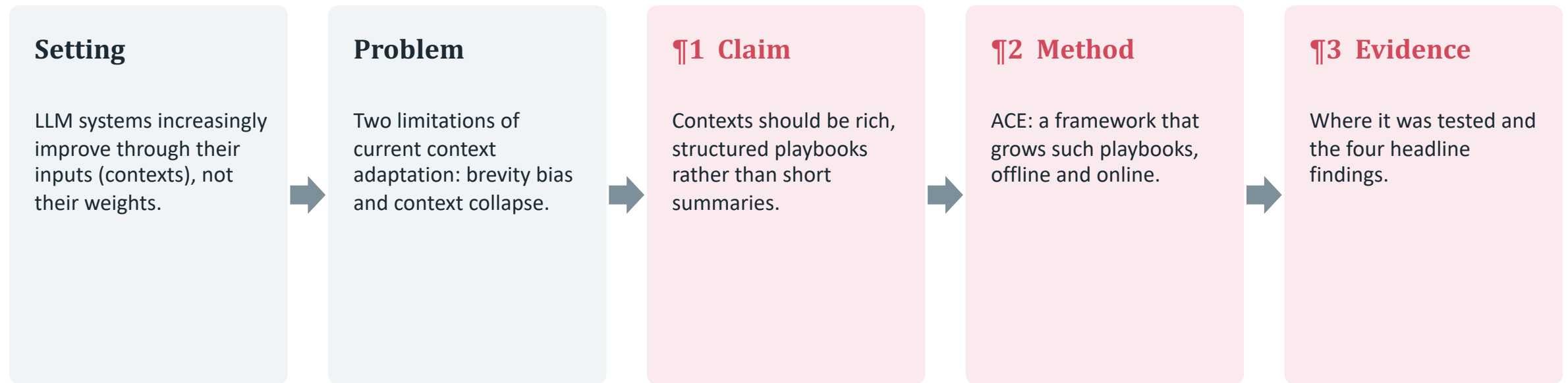
The last three paragraphs: the claim, the method, the evidence

Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models · ICLR 2026

Paragraphs beginning “As applications...”, “To address these limitations...”, and “We evaluate ACE...”

Where these three paragraphs sit in the argument

The introduction builds one line of reasoning; the last three paragraphs are claim → method → evidence



What this means

The first half of the introduction sets up a problem; ¶1 states what a good context should look like, ¶2 introduces the framework that builds it, and ¶3 previews the experiments that support it. Reading ¶2 requires the “limitations” from the preceding paragraph, explained on the next slide.

Background: the two limitations that ¶2 responds to

Defined in the paragraph just before ¶1; “To address these limitations” in ¶2 refers to these

Brevity bias

Many prompt optimizers reward short, general instructions. Compression throws away the specific heuristics, tool-use guidelines and known failure modes that agents and expert tasks rely on.

Example given: GEPA presents brevity as a strength.

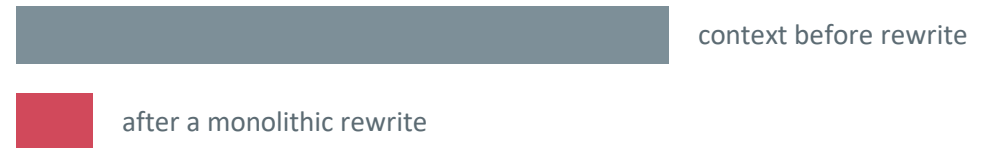
What this means

Both problems lose knowledge: one by design (asking for short prompts), one by accident (repeated rewriting). ACE is designed to avoid both.

Context collapse

Methods that let an LLM rewrite the whole context at each step tend to shrink it into ever shorter, less informative summaries, and accuracy drops sharply when that happens.

Shown in the paper’s Figure 2: a large context abruptly rewritten into a tiny one.



The claim: a context should be a playbook, not a summary

“As applications like agents ...”

Paragraph 1

What the paragraph says

- 1 Agents and knowledge-heavy reasoning need reliability, so recent work fills contexts with more potentially useful information; long-context models make this feasible.
- 2 The authors take a position: contexts should be “comprehensive, structured playbooks”: detailed, inclusive, and full of domain insight.
- 3 Why: people often do better with a concise rule of thumb, but LLMs tend to do better with long, detailed contexts and can work out for themselves which parts matter.
- 4 So: keep domain heuristics and tactics in the context instead of compressing them away, and let the model decide what is relevant at inference time.

Summary-style context

“Classify stress carefully.”

Short, general, human-friendly.
Specific tactics and pitfalls are gone.

Playbook-style context

- strategy: watch indirect signs
- rule: denial ≠ no stress
- pitfall: topic words alone mislead
- ...dozens more, each with a track record

Long and specific; the model selects what applies.

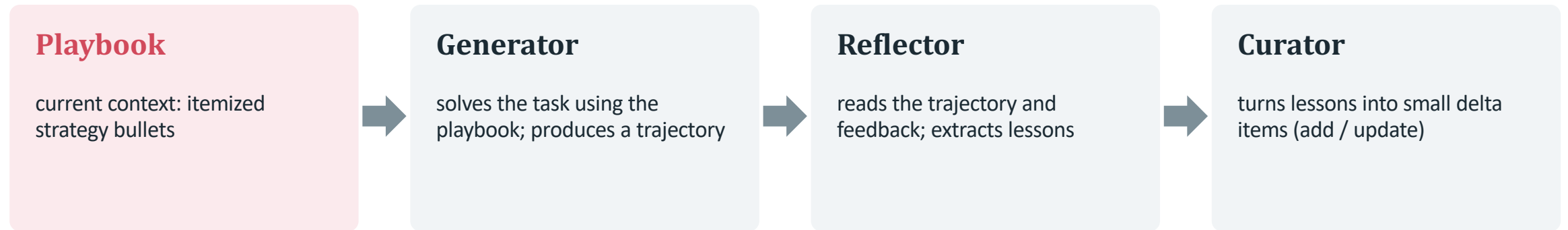
What this means

The paragraph flips the usual intuition that shorter prompts are better. Its premise is empirical: LLMs can use long contexts well, so detail is an asset rather than noise.

The method: ACE grows the playbook step by step

“To address these limitations, we introduce ACE ...”

Paragraph 2



↪ delta items are merged back into the playbook by simple, non-LLM logic; the playbook is never rewritten wholesale

Two settings

Offline: optimize the system prompt before deployment.
Online: adapt a memory at test time, item by item.

Evolving playbook

Strategies accumulate and are organized over time instead of being condensed into a summary.

Grow-and-refine

Structured, incremental updates add new items and refine existing ones, so detail is kept and collapse is avoided.

What this means

The key design choice is updating the context in small, itemized pieces: nothing is summarized away (brevity bias) and nothing is rewritten wholesale (context collapse).

The evidence (1/2): two kinds of application

“We evaluate ACE on two categories of LLM applications ...”

Paragraph 3

Agents

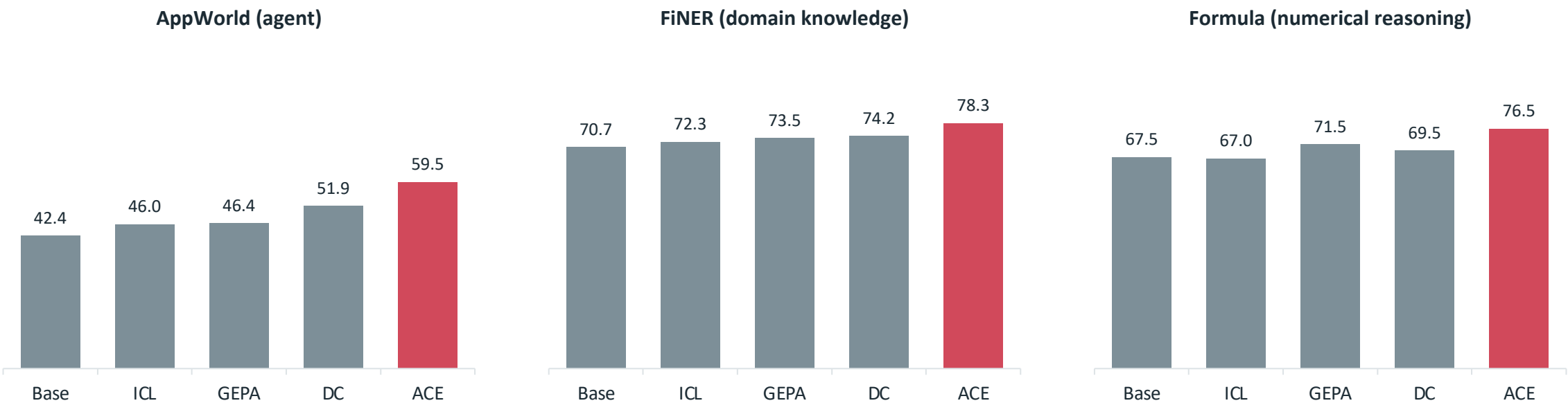
Multi-turn reasoning, tool use and interaction with an environment. Strategies learned in one episode can be reused in the next.

Benchmark: AppWorld (apps driven through code and APIs)

Domain-specific benchmarks

Tasks that need specialized tactics and knowledge, such as financial analysis.

Benchmarks: FiNER and Formula (finance)



Accuracy (%) as reported in the paper’s Figure 1 (DeepSeek-V3.1 backbone); AppWorld values average the four TGC/SGC scores. Source: Zhang et al., “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models”, ICLR 2026 (arXiv 2510.04618), Section 1.

The evidence (2/2): four headline findings

What the paragraph promises the experiments will show

Paragraph 3

+10.6% / +8.6%

Consistent gains

Average improvement over strong baselines on agents and on domain-specific benchmarks, in both offline and online settings.

No labels needed

Learns from feedback

Effective contexts can be built from execution feedback and environment signals instead of labeled answers, a key ingredient for self-improvement.

Leaderboard

Open model, top-tier agent

On the AppWorld leaderboard, ACE with open-source DeepSeek-V3.1 surpasses the top-ranked production agent (IBM-CUGA, built on GPT-4.1).

Cheaper

Fewer rollouts, lower latency

ACE needs fewer rollouts and adapts faster than existing adaptive methods, so the gains do not come at higher cost.

Terms used in these paragraphs

Short definitions to keep at hand while reading

Context adaptation

Improving a model by changing its inputs (instructions, memory, evidence) instead of its weights.

Playbook

ACE's context: a growing, itemized list of strategies, rules and pitfalls, each with usage counters.

Brevity bias

The tendency of prompt optimizers to favour short instructions, losing detailed knowledge.

Context collapse

Loss of information when an LLM repeatedly rewrites the whole context into a shorter one.

Delta update

A small, itemized change (add or edit one bullet) merged into the playbook without a rewrite.

Grow-and-refine

Keep adding new items while periodically refining (merging or de-duplicating) existing ones.

Offline / online

Adapt before deployment on training data, or adapt during use, one test item at a time.

Execution feedback

Signals from running the task (errors, tool outputs, test results) used instead of labels.

Rollout

One full attempt at a task by the model; fewer rollouts means cheaper adaptation.

How the claims compare with what we see (Qwen3-8B)

Internal working observations from our v5 panel and the AppWorld reproduction, both complete; not manuscript evidence

Claim in these paragraphs	What we observe	Status
Contexts should be detailed and well organized (¶1–2)	Dreaddit playbooks are detailed (36–63 bullets) but poorly organized: 198 of 200 bullets sit in one section and one formula repeats nine times (refine step off).	Partly
ACE beats strong baselines (¶3, finding 1)	On Dreaddit all four ACE conditions beat Base by 7.0–11.3 points, but GEPA (+9.7) is close: it beats both ACE conditions without labels and trails the best ACE condition by 1.7.	Partly
No labels needed (¶3, finding 2)	Dreaddit ACE without labels: +7.3 (offline) and +7.0 (online) points over Base.	Consistent
Agents benefit most (¶3)	AppWorld with only the model swapped: offline ACE beats ReAct on test-normal (TGC 22.0 vs 17.9, SGC 10.7 vs 3.6), but the gain is a third of the paper’s and every score is far below it.	Partly
Fewer rollouts, lower cost (¶3, finding 4)	Not measured in our setup.	Not tested

What this means

With an 8B model the direction of the headline claims holds on both benchmarks — ACE beats its baselines, and labels are not required — but the size of the gains does not, and a strong prompt optimizer (GEPA) comes close on Dreaddit. In both cases the playbooks look nothing like the organized, multi-section documents the paper describes: one section absorbs everything and duplicates pile up. Organization and deduplication are where a smaller backbone seems to differ.

Our numbers: v5 server panel, Qwen3-8B, Dreaddit test set (100 posts × 3 seeds). AppWorld: ace-appworld paper-release code with only the LLM replaced.